

This diagram is generated from the EF Core model in `src/LearnHub/Models` and the Fluent API configuration in `src/LearnHub/Data/Configurations`. Table names are EF Core defaults, which match the `DbSet` names in `src/LearnHub/Data/ApplicationDbContext.cs`. There is one provider (SQLite), one context and one migration: `Data/Migrations/20260914100832_InitialCreate.cs` creates all **18 tables** — the eleven application tables below plus the seven ASP.NET Core Identity tables.

[illegible]

Page 1 of 3

Relationships and delete behaviour

Parent → child	Cardinality	On delete	Why
Categories → Courses	1 : many	Restrict	Deleting a category must never delete courses silently. The admin delete page explains that the category still has courses.
Courses → LearningResources	1 : many	Cascade	A course owns its lessons.
Courses → Quizzes	1 : many	Cascade	A course owns its quizzes.
Courses → Enrollments	1 : many	Cascade	Enrolments are meaningless without the course.
AspNetUsers → Enrollments, ResourceCompletions, QuizAttempts	1 : many	Cascade	Deleting an account removes that person's learning records.
LearningResources → ResourceCompletions	1 : many	Cascade	Completion records belong to the lesson.
Quizzes → Questions → AnswerOptions	1 : many	Cascade	Questions and options cannot exist without their quiz.
Quizzes → QuizAttempts → QuizAnswers	1 : many	Cascade	Attempts belong to the quiz.
Questions → QuizAnswers	1 : many	Restrict	A second cascade path (Quiz → Question → QuizAnswer beside Quiz → QuizAttempt → QuizAnswer) would let a quiz edit delete a student's answer history silently, so the link restricts instead.
AnswerOptions → QuizAnswers	0..1 : many	Restrict	Same reason; an answer may also be empty (unanswered question).

Because of the two Restrict rules, `CourseManagementService`, `QuizManagementService` and `QuestionManagementService` delete the affected `QuizAnswers` first with `ExecuteDeleteAsync`, then the parent, inside one transaction created through the provider's execution strategy (`Data/TransactionExtensions.cs`).

Keys, indexes and constraints

Table	Unique	Other indexes	Check constraints
AspNetUsers	NormalizedUserName (<code>UserNameIndex</code>); email uniqueness enforced by Identity (<code>RequireUniqueEmail</code>)	NormalizedEmail (<code>EmailIndex</code>)	–
AspNetRoles	NormalizedName (<code>RoleNameIndex</code>)	–	–
Categories	Name	–	–
Courses	–	CategoryId; Title; (IsPublished, CategoryId)	<code>CK_Courses_DurationMinutes</code> : DurationMinutes > 0

Table	Unique	Other indexes	Check constraints
LearningResources	–	(CourseId, SortOrder); (CourseId, IsPublished)	CK_LearningResources_SortOrder : SortOrder >= 0
Enrollments	(UserId, CourseId)	(CourseId, CompletionPercentage)	CK_Enrollments_CompletionPercentage : 0–100
ResourceCompletions	(UserId, LearningResourceId)	LearningResourceId	–
Quizzes	–	CourseId	CK_Quizzes_PassMarkPercent : 0–100
Questions	–	(QuizId, SortOrder)	CK_Questions_Points : 1–100
AnswerOptions	–	(QuestionId, SortOrder)	–
QuizAttempts	–	(UserId, QuizId); CompletedAt; QuizId	CK_QuizAttempts_ScorePercent : 0–100; CK_QuizAttempts_CorrectCount : 0 ≤ CorrectCount ≤ QuestionCount; CK_QuizAttempts_Score : 0 ≤ Score ≤ MaxScore
QuizAnswers	(QuizAttemptId, QuestionId)	QuestionId; SelectedOptionId	–
ContactMessages	–	(IsRead, CreatedAt)	–

The unique indexes are the last line of defence against double-submitted forms: for example, if two requests enrol the same student at the same moment, the second insert fails on (UserId, CourseId) and EnrollmentService turns the DbUpdateException into the message "You are already enrolled in this course."

Design notes

- **Timestamps.** Entities implementing IHasTimestamps get CreatedAt and UpdatedAt set in ApplicationDbContext.SaveChanges. A UtcDateTimeConverter value converter stores every DateTime as UTC, so the stored values do not depend on the machine's time zone; the browser converts them for display.
- **Progress is stored, then kept honest.** ProgressService recalculates Enrollments.CompletionPercentage and Enrollments.CompletedAt as (published lessons completed + passed published quizzes that have questions) ÷ (published lessons + published quizzes that have questions). Caching it keeps the dashboard, the student pages and the admin lists to one query, and the recalculation runs whenever a lesson is completed, a quiz is passed, an enrolment changes or course content is added or removed — so adding a lesson correctly lowers everyone's percentage.
- **Score snapshots.** QuizAttempts stores Score, MaxScore, CorrectCount, QuestionCount, ScorePercent and Passed, and each QuizAnswers row stores IsCorrect, so results stay truthful even if the quiz, its points or its options are edited later. Questions.Points weights the questions, and the percentage is rounded down.
- **Deactivation without a flag.** A deactivated user has LockoutEnd set to the maximum date. Identity already refuses sign-in for locked-out users, so no extra IsActive column can drift out of sync.
- **Leaving a course.** Removing an enrolment keeps completion records and quiz attempts, so progress returns if the student enrolls again.
- **Files.** Uploaded files are stored outside wwwroot; the database keeps only the generated file name, the original file name, the detected content type and the size.